INTRODUCTION TO GIT/GITHUB

GITHUB| GIT

FACILITATOR
Kolawole David
Olanipekun

INTRODUCTION TO GIT & GITHUB

GIT

Git is a distributed version control system that lets developers track changes in their codebase, collaborate with others, and revert to earlier versions if needed. It works locally on your computer and is used to manage source code efficiently—especially in team environments.Created by Linus Torvalds (creator of Linux).Key commands: git init, git add, git commit, git push, git pull, etc.

GITHUB

GitHub is a web-based platform that hosts Git repositories in the cloud. It provides tools for collaboration, issue tracking, pull requests, and version control. GitHub acts as a remote server where you can store your Git projects and work with others globally.

- Owned by Microsoft.
- Enables teams to manage projects, review code, and automate workflows (e.g., using GitHub Actions).

GitHub Desktop

GitHub Desktop is a GUI (Graphical User Interface) application that simplifies working with Git and GitHub without using the command line. It allows users to clone repositories, make commits, create branches, and push/pull code—all through a user-friendly interface.

- Great for beginners or those who prefer not to use the terminal.
- Available for Windows and macOS.

INTRODUCTION TO GIT & GITHUB

GitHub Setup

Step 1: Install Git on Windows

1. Download Git for Windows:
   - Go to: https://git-scm.com/download/win
   - The download should start automatically.
2. Run the Installer:
   - Double-click the downloaded .exe file.
   - Use the default settings unless you know what to change. Click Next on each screen.

Once installed, search for “Git Bash” in the Start Menu and open it to use Git.

INTRODUCTION TO GIT & GITHUB

Step 2: Create a GitHub Account (if you haven't already)

Go to: https://github.com/

Click Sign up, enter your email, password, and username, then verify your email.

Step 3: Set Up Git with GitHub

Open Git Bash and configure your user:

git config --global user.name "Your Name"

git config --global user.email "you@example.com"

Replace Your Name and you@example.com with your actual GitHub name and email.

COMMON COMMANDS IN GIT

🛠️ 1. Setup & Configuration

1. `git config`

   Sets Git configuration values (user info, editor, aliases).

   Example:

   `git config --global user.name "Your Name"`

2. `git init`

   Initializes a new Git repository in the current directory.

   Example:

   `git init`

3. `git clone`

   Creates a local copy of a remote repository.

   Example:

   `git clone https://github.com/user/repo.git`

📁 4. Basic Workflow

4. `git status`

   Shows the current status of the working directory and staging area.

5. `git add`

   Adds file(s) to the staging area (prepares for commit).

   Example:

   `git add filename.py` or `git add .` (adds all files)

6. `git commit`

   Records staged changes with a message.

   Example:

   `git commit -m "Initial commit"`

7. `git push`

   Uploads local commits to the remote repository.

   Example:

   `git push origin main`

8. `git pull`

   Fetches and merges changes from a remote repository.

   Example:

   `git pull origin main`

9. `git fetch`

   Downloads changes from a remote repository w ut merging.

COMMON COMMANDS IN GIT

🌿 10. Branching

10. `git branch`

    Lists, creates, or deletes branches.

    Example:

    `git branch new-feature`

11. `git checkout`

    Switches to a different branch or commit.

    Example:

    `git checkout dev`

12. `git switch`

    Simplifies branch switching.

    Example:

    `git switch main`

13. `git merge`

    Merges changes from one branch into another.

    Example:

    `git merge dev`

14. `git rebase`

    Applies commits from one branch onto another, linearly.

📜 15. History & Logs

15. `git log`

    Shows commit history.

    Example:

    `git log --oneline`

16. `git show`

    Displays detailed information about a specific commit.

    Example:

    `git show <commit-hash>`

17. `git diff`

    Shows changes between commits, branches, or working directory.

    Example:

    `git diff main dev`

COMMON COMMANDS IN GIT

18. Undoing Changes

18. `git reset`

    Unstages changes or resets commits.

    Example:

    `git reset --hard HEAD~1`

19. `git revert`

    Reverses a specific commit by creating a new one.

20. `git clean`

    Removes untracked files from the working directory.

    Example:

    `git clean -f`

21. Remote Repos

21. `git remote`

    Manages remote repositories.

    Example:

    `git remote add origin https://github.com/user/repo.git`

22. `git remote -v`

    Lists the remotes with URLs.

23. Tags

23. `git tag`

    Creates or lists tags (for releases).

    Example:

    `git tag v1.0`

24. `git push --tags`

    Pushes tags to the remote repository.

COMMON COMMANDS IN GIT

🪥 25. Stashing & Temporary Work

25. `git stash`

    Temporarily saves uncommitted changes.

    Example:

    `git stash save "WIP"`

26. `git stash pop`

    Reapplies the stashed changes and removes them from stash.

27. `git stash list`

    Shows list of all stashed changes.

📒 28. Collaboration

28. `git cherry-pick`

    Applies a commit from one branch into another.

    Example:

    `git cherry-pick <commit-hash>`

29. `git blame`

    Shows which commit and author last modified each line of a file.

30. `git shortlog`

    Summarizes git log output grouped by author.